Unit 4 Experimentation Platform and Cultures.

(1) Experimentation Maturity Models.

-> An Experimentation Maturity Model is a framework that describes how advanced an organisation is at running online experiments (like A/B tests).

-> It covers both the technical infrastructure and the cultural mindset toward using data-driven decision-making.

(1) Goal of the Maturity Model.

To evaluate and improve how well a company:

- Designs
- Implements
- Analyzes
- Trusts and acts upon experiment

This is important because just running A/B tests isn't enough - you need quality, trust, scale, and culture to extract real value.

(2) Maturity levels (like growing from baby to a professional athlete).

Imagine a company's experimentation journey like a person growing in fitness training.

- **Level 1 : Manual Testing (Infant stage)**

-> Analogy: Like jogging without a fitness watch.

-> Teams do ad-hoc testing manually. No infrastructure
-> Data is often manually analyzed in Excel.
-> High chance of bias, errors, or p-hacking.

• Level 3: ~~Basic Infrastructure (Teenager Stage)~~ Centralized platform (college Athlete)

-> Analogy: Uses smartwatch, heart rate monitor, training coach.
-> Unified platform for experimentation (like Netflix's XP, Microsoft's ExP, Google's Opt Experiment).
-> Logging, analysis, and telemetry are consistent.
-> Culture starts trusting experiments more.

• Level 3: Centralized Platform (college Athlete)

-> Analogy: Uses smartwatch, heart rate monitor, training coach.

• Level 2: Basic Infrastructure (Teenager Stage)

-> Analogy: Jogging with a stopwatch and apps

-> A/B tools start to appear, but different teams use different systems.

-> Still not scaleable or trustworthy.

-> Some governance begins.

Level 4 : Self-Service Scalable Platform (Pro Athlete).

-> Analogy : Personal data dashboard, AI-Coach, live feedback.
-> Teams launch and analyze experiments independently and safely.
-> Guardrails ensure that bad experiments don't harm customers.
-> Diagnostics automatically detect bias, SRM (sample ratio mismatch), and other trust threats.

Level 5 : Experimentation-Driven Culture (Olympic Champion)

-> Analogy : Lives and breathes data

-> Everyone - product, marketing, engineering - make all major decisions through experiments.
-> There is a culture of learning from failures and iterative experimentation.
-> Experiments are done even on internal tools, documentation etc.

3 (b) Math Behind the Maturity : Statistical Trust Grows with Maturity.

Here's the catch : The math of statistical inference doesn't change, but the way a company applies it becomes safer and more trustworthy at higher maturity levels.

→ low-maturity orgs often misinterpret p-values, cherry-pick data, or ignore confidence intervals.

→ High-maturity orgs use pre-registration, understand statistical power, detect SRM, and monitor Type I/II errors actively.

(4) Real-world Analogy in Code: Fitness Trackers vs. Random Experiment.

Let's simulate a maturity journey using Python – from manual testing to platform-powered analysis.

We'll simulate:

1. A/B test with no guardrails (Level 1)
2. Same A/B test with automatic diagnostics (Level 4+)

Manual A/B Test (Level 1 – Risky).

| ⚠ No diagnostics. No SRM check. Risky interpretation.

Mature A/B Test (Level 4 – Diagnostics Included).

| ☑ A mature platform automatically flags SRM, summarizes conversion rates, and shows rollback status.

(5) Key Takeaways.

Maturity level	Trust level	Culture	Tooling
Level 1	low	Skeptical or unaware	Manual excels, random scripts.
Level 3	Medium	Curious	Unified dash boards
Level 5	High	Data-first, experiment-first	Scalable, automated, safe.

⑥ Final Thought.

→ The Experimentation Maturity Model isn't just a technical ladder – it's trust ladder. As maturity increases.

- The math stays the same but
- The confidence in insights grows.
- And errors shrink, as the platform and culture catch them early.

② Infrastructure and Tools.

① What is "Infrastructure and Tools" in the Context of A/B Testing?

→ This refers to the technical systems and software tools required to:

- Design experiments
- Deliver treatments (variations)
- Track users and their behaviour
- Collect data safely.
- Run analysis with guardrails.
- Monitor results and roll backs.

Without strong infrastructure, experiment are error-prone, slow and hard to trust.

② Why does Infrastructure Matter?

→ Experiments are statistical studies running on real users. Poor infra means:

- wrong users in the wrong bucket.
- Biased logging or corrupted metrics.
- Undetected SRM (sample Ratio Mismatch).
- Invalid inference

Think of infrastructure as the "plumbing" of experimentation. If the pipes leak, the data is dirty — no matter how clever your statistics are.

(3) Core Components of Experimentation Infrastructure

1. Experimentation Service (Randomization Engine)

Function: Assigns users to control or treatment

Analogy: Decide which customers get the old burger vs. new burger.

Math concept:
Randomization = ensures independence → prevents selection bias.

(1) Experimentation Service (Randomization Engine)

Function: Assigns users to control or treatment.

```
Import random
def random_assign (user_id, traffic_split =0.5):
    random.seed (user_id) # deterministic assignment
    return "treatment" if random.random() < traffic_split
    else "control"
```

☑ if this fails (e.g. assigns 70% to treatment), your experiment is biased (violates internal validity)

② Exposure logging (Telemetry)

→ Exposure logging is the process of recording when a user is actually shown a variant (control or treatment) in an experiment.

⚠ It is not enough to assign someone to a group - we must log if they were actually exposed to the feature.

Real-world Analogy

Think of a cinema test screening.

- You give 1000 people tickets (random assignment)
- But only 800 actually enter the hall and see the movie (exposure).
- You ask everyone about the movie's quality.

If you include all 1000 — you're lying to yourself. Only 800 actually watched it, so you must log and analyze based on exposure, not assignment alone.

③ Metric Definition Layer (Standardized KPIs)

-) Functions: Converts raw data into business metrics (e.g. conversion rate, revenue, churn).

-) Analogy: Truck: Did the customer finish the burger? Buy more?

```
exposure_log = []
for user in range(10):
    group = random.assign(user)
    exposure_log.append({"user": user, "group
```

~~df["converted"]~~

```
df2 = pd.DataFrame(exposure_log)
df["converted"] = df["group"].apply
    (lambda g: 1 if g == "treatment"
     and random.random() < 0.12
     else (1 if g == "control" and
     random.random() < 0.10 else 0))
conversion_rate = df.groupby("group")["converted"]
    .mean().
```

well-defined metrics reduce ambiguity, enable reproducibility, and stop cherry-picking.

④ Guardrails and Diagnostics

Function: Auto-checks for issues.

- SRM (Sample Ratio Mismatch)
- Missing data
- Outliers
- Violations of assumptions.

Overlaps with "Misinterpretation of Statistical Results" & SRM.

```
def check_srm(df):
    expected = len(df)/2
    observed_control = (df['group']=='control').sum()
    return abs(observed_control - expected)/expected > 0.05
```

check_srm(df) # Returns True if SRM is detected.

⑤ Dashboard and Visualization Tools.

Functions: Let teams view, interpret, and act on results.

Analogy: Manager's dashboard showing "burger satisfaction" starts over time.

Modern platforms like:

- Airbnb's XP
- Netflix XP
- Uber's uMetric
- Microsoft's ExP.

... all offer dashboards where product managers can:

- launch experiments
- view p-values, CI
- Detect issues
- Stop / continue experiments.

(4) Why This Matters: Garbage In, Garbage out.

> Even the best statistical models are useless without clean input.

Experimentation infrastructure is the foundation of "Trustworthy experimentation"

If your infrastructure

- Has leaky logging → metrics are false
- Has bad randomization → causality breaks
- Lacks diagnostics → wrong decisions.

Then even a perfect p-value doesn't save you.

(5) Inter linked Topics!

Concept — Linkage.

SRM Detection — Depends on randomization & logging

Threats to Internal Validity — Guardrails in infrastructure detect

Confidence Intervals	Calculated once ~~not~~ metrics are properly defined.
Experimentation Maturity Model.	Maturity grows with richer infrastructure and self-service tools.

(8) Final Summary

Layer	Descriptions	Core Risk if Missing
Randomization	Assigns users fairly	Biased inference
Logging	Track what user saw	Incorrect exposure
Metric Definitions	Converts logs - insights	Garbage metrics.
Guardrails	Catch common issues	Trust breakdown
Dashboard	Empower stakeholder	Slow decisions.

dhanashreedighising